[T2.2] K=1

- **Algoritmo**

1. Construir o grafo bipartido (Source → Revisores → Artigos → Sink).
2. Calcular o **Fluxo Máximo original** (FluxoEsperado) usando Edmonds-Karp.
3. Para cada revisor R:
    - Remover R(definir capacidade da aresta Source → R = 0).
    - Recalcular o fluxo máximo (FluxoAtual).
    - Se **FluxoAtual < FluxoEsperado**, então R é **crítico**.
4. Restaurar o grafo e continuar.

- **Complexidade**:

    - Iteramos por todos os N revisores
    - Cada execução de fluxo: **O(V · E²)**
    - Total: **O(N × V · E²)**

Exemplo de Entrada Não Trivial

Um exemplo não trivial onde a capacidade total do sistema é **superior** à necessária, mas a **especialização técnica** cria um ponto único de falha.

Configuração do Exemplo:

- **Submissões (S):** 3 submissões (S1, S2, S3).
    - {S1, S2, S3} tema "Criptografia".
- **Revisores (R):** 4 revisores (R1, R2, R3, R4).
    - R1, R2: Especialistas em “AI”.
    - R3, R4: Especialistas em “Criptografia”.
- **Restrições:** Cada submissão precisa de **2 revisões**. Cada revisor pode fazer no máximo **3 revisões**.
- **Fluxo esperado:** 3 x 2 = 6 unidades de fluxo
- **Capacidade Total:** 12 vagas (o dobro do necessário).

Análise do Resultado (Output):

1. **Estado Normal:** O fluxo máximo é 6. As submissões de Criptografia são atendidas por R3 e R4 (3 + 3 = 6).
2. **Falha de R3:** Ao zerar a capacidade de R3, a capacidade disponível para Criptografia cai para **3** (apenas R4).
3. Como as submissões precisam de **6** revisões no total, o fluxo máximo cai para 3.
4. **Conclusão:** O algoritmo identifica R3 (e R4) como **Revisores Críticos**, apesar de o sistema ter 6 vagas "de sobra" em AI que não podem ser utilizadas.

Output Exemplo:

```
#Risk Analysis: 1

3, 4
```

[T2.3] K>1

O que significa:

- K>1 significa que queremos encontrar **conjuntos de K revisores** cuja **remoção reduz o fluxo máximo**.
- Objetivo: identificar **os pontos de falha do sistema** em termos de capacidade de revisão.

1. Abordagem por Brute Force

A estratégia inicial consiste em testar todas as combinações possíveis de revisores no sistema. Para um conjunto de N revisores totais, o algoritmo gera todas as combinações de tamanho K.

- **Algoritmo:**

1. Construímos o grafo bipartido.
2. **Calcular Max-Flow** no grafo completo.
3. Gerar todas as combinações C(N,K).
4. Para cada combinação, remover os K revisores do grafo (capacidade da aresta Source →Rev = 0).
5. Recalcular o Fluxo Máximo utilizando Edmonds-Karp.
6. Se FluxoAtual < FluxoEsperado o conjunto é marcado como crítico.

- **Complexidade:** O(C(N,K) × (V. E²)).
- **Limitação:** Esta abordagem torna-se computacionalmente inviável à medida que N ou K crescem, devido à explosão combinatória.

2. Otimização via Min-Cut

Para otimizar o processo, uma melhor abordagem seria através do teorema **Max-Flow Min-Cut**. Identificamos que apenas os revisores que pertencem ao "bottleneck" do fluxo (o corte de menor capacidade que separa a *Source* do *Sink*) têm o potencial de reduzir o fluxo total se removidos.

Nota: **Min-Cut não identifica todos os críticos**, apenas candidatos que podem impactar o fluxo.

- **Algoritmo:**

1. Construímos o grafo bipartido.
2. **Calcular Max-Flow** no grafo completo.
3. **Calcular Min-Cut** para filtrar revisores candidatos (lado da Source).
4. **Gerar todas as combinações de K revisores candidatos**.
5. Para cada combinação, remover os K revisores do grafo (capacidade da aresta Source →Rev = 0).
6. Recalcular o Fluxo Máximo utilizando Edmonds-Karp.
7. Se FluxoAtual < FluxoEsperado o conjunto é marcado como crítico.

- **Complexidade:**
  - Edmonds-Karp por execução: $O(V \cdot E^2)$
  - Testar combinações K: $O(C(r,K) \times (V. E^2))$, onde r = número de revisores candidatos do Min-Cut .

Exemplo de Entrada Não Trivial

Para demonstrar a eficácia da otimização, consideremos o seguinte cenário:

6.1. Configuração do Cenário

- **Submissões (S):** 4 submissões (S1, S2, S3, S4).
  - {S1,S2,S3} tema “AI”.
  - {S4} tema “Cibersegurança”.
- **Revisores (R):** 5 revisores (R1, R2, R3, R4, R5).
  - {R1,R2,R3} especialistas em “AI”.
  - {R4,R5}especialistas em “Cibersegurança”.
- **Restrições:** Cada submissão precisa de **2 revisões**. Cada revisor pode fazer no máximo **2 revisões**.
- **Fluxo Esperado:** 4 x 2 = 8 unidades de fluxo.

- **O Gargalo:** Embora tenhamos capacidade total de 10 (5 x 2), as submissões S1, S2, S3 precisam de 6 revisões no total, e dependem exclusivamente de R1, R2, R3 (cuja capacidade somada é exatamente 6).

6.2. Execução e Output Esperado (K=2)
Passo 1: Max-Flow Inicial

- **Resultado:** 8

Passo 2: Identificação do Min-Cut (Otimização)

- O algoritmo identifica que as arestas Source -> R1, R2, R3 estão **saturadas** e limitam o fluxo das primeiras submissões.
- **Candidatos Críticos (r):** {R1, R2, R3}.
- *Nota:* R4 e R5 não entram no Min-Cut pois têm "folga" atendendo apenas S4.

Passo 3: Teste de Combinações (K=2)

- **Brute Force:** Testaria C(5,2) = 10 combinações.
- **Otimizado:** Testa apenas C(3,2) = 3 combinações: {R1, R2}, {R1, R3}, {R2, R3}.

Output Exemplo:

```
#Risk Analysis: 2

{R1, R2}, {R1, R3}, {R2, R3}
```

[T2.4] General Formulation without Risk Analysis

1. Abordagem: Expansão da Rede de Fluxo

A formulação geral (Mode 3) expande o problema base ao permitir que tanto revisores como submissões possuam domínios de especialidade secundários. A nossa solução utiliza o algoritmo de **Max-Flow** numa rede bipartida com as seguintes características:

- **Nós e Arestas de Carga:** Mantemos a *Super-Source* ligada aos revisores (R_i) com capacidade MaxReviews e as submissões (S_j) ligadas ao *Super-Sink* com capacidade MinReviews.
- **Conetividade de Especialidade:** No Modo 3, o grafo torna-se muito mais denso. Uma aresta de capacidade 1 é criada entre o Revisor i e a Submissão j se ocorrer qualquer uma das 4 combinações de match:
    1. Revisor Primário == Submissão Primária
    2. Revisor Primário == Submissão Secundária
    3. Revisor Secundário == Submissão Primária
    4. Revisor Secundário == Submissão Secundária
- **Resolução:** O algoritmo de Edmonds-Karp percorre esta rede para encontrar o emparelhamento que satisfaça todos os requisitos de revisão simultaneamente.

2. Análise de Complexidade

O caso geral é computacionalmente mais exigente do que o caso base:

- **Construção do Grafo:** O(R x S). O algoritmo testa as 4 combinações de domínios para cada par revisor-submissão.
- **Densidade do Grafo:** O número de arestas (E) no Modo 3 pode ser até 4 vezes superior ao Modo 1. No pior cenário (revisores universais), E ~= R x S.
- **Edmonds-Karp:** O(V. E^2), Dado que E aumenta drasticamente nesta formulação, o tempo de execução cresce de forma quadrática em relação ao número de arestas, tornando esta a tarefa mais pesada fora da análise de risco.